

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT 1 DRAFT REPORT

FOUNDATIONS OF DEEP LEARNING
PIPELINES AND ARCHITECTURES

From Linear Models to Modern Sequence Models: A Comparative Study for Image
Classification

Course: CO3133 – Deep Learning and Its Applications

Semester: 261

Instructor: Lê Thành Sách

Class: CC01

Group: Tiki-taka

Name	Student ID
Tạ Tuấn Khải	2452515
Nguyễn Hạo Thiên	2453194
Nguyễn Anh Khoa	2452539

Ho Chi Minh City, September 2026

ASSIGNMENT 1 CONTRIBUTIONS

The following records the contributions reported for this draft. The entries remain member-reported until the group completes its submission review.

Member	Reported contribution
Nguyễn Hạo Thiên (2453194)	Data preparation, EDA, stratified split, transforms/loaders and Linear model; data and figure review.
Nguyễn Anh Khoa (2452539)	Shared training engine, metrics, checkpoints and MLP; selected training runs, analysis and Optuna exploration.
Tạ Tuấn Khải (2452515)	Configuration, reproducibility/artifacts, integration, analysis tooling and report/Pages editing.

Draft results cover Linear and MLP. CNN, recurrent and Transformer experiments are planned for the Final milestone.

Table of Contents

Draft Scope and Evidence Status	1
1 Part 1 — Problem and Data Description	3
1.1 Problem Formulation	3
1.2 Dataset Identity and Scope	3
1.2.1 Collection, Labeling, and Processing Methodology	3
1.2.2 Acquisition, Formats, and Measured Storage Size	4
1.3 Exploratory Data Analysis	5
1.4 Split and Leakage Controls	7
1.5 Preprocessing and Augmentation	8
1.6 Dataset and DataLoader Design	9
2 Part 2 — Methodology	11
2.1 Shared Pipeline and Implementation Boundaries	11
2.2 Interfaces and Representations	11
2.3 Linear Classifier	11
2.4 Multilayer Perceptron	12
2.5 Loss and Metrics	13
2.6 Training and Validation	13
2.7 Actual Selected-Run Configurations	13
2.8 Checkpoints, Configuration and Provenance	14
2.9 Fairness and Test Policy	15
2.10 Randomness and Environment	15
2.11 Efficiency and Analysis Method	16
3 Part 3 — Implementation Results and Draft Discussion	17
3.1 Verification Record	17
3.2 Selected Run Inventory	18
3.2.1 Adjusted-Run Revision Qualification	18
3.3 Learning Curves	19
3.4 Quantitative Results	21
3.4.1 Forward-Only Inference Measurements	21
3.5 Qualitative Results	21



3.6	Error Analysis and Representation	22
3.7	Comparison and Discussion	24
3.8	Implementation Issues and Corrections	25
3.9	Tuning Context and Additional Runs	25
3.9.1	Short-Budget Optuna Exploration	25
3.9.2	Tuning Qualifications and Archive Reconciliation	26
3.10	Limitations	27
3.11	Draft Conclusion	28
3.12	Plan Toward Final	28
3.13	Reproducibility and Artifact Access	28
3.14	Contributions and AI Disclosure	30
	References and Evidence	32
	A Companion Confusion Matrices and Error Galleries	33
	References	37

Abstract

This draft develops a shared PyTorch pipeline for Fashion-MNIST image classification and compares a Linear classifier with a multilayer perceptron (MLP). The pipeline uses a saved stratified partition, train-only normalization, explicit train/validation behavior, validation-based checkpoint selection and versioned experiment artifacts. Four selected saved runs, comprising baseline and adjusted configurations for each model, are reported at run seed 69420. Baseline validation accuracy is 79.16% for Linear and 90.15% for MLP. The adjusted runs achieve 80.09% and 89.51%, respectively. Recalculation from 10,000 exported validation predictions per run agrees with the saved metrics. Confusion analysis identifies upper-body garment distinctions as a prominent remaining difficulty. Supplied Tesla T4 forward-only measurements at batch size 64 show a median batch time of 0.068305 ms for baseline Linear and 0.177286 ms for baseline MLP. These single-seed results and single-session timing records do not establish across-seed uncertainty, deployment latency or isolated hyperparameter effects. CNN, recurrent and Transformer experiments are reserved for the Final milestone.

Keywords: Fashion-MNIST; image classification; Linear classifier; MLP; reproducibility; validation.

Draft Scope and Evidence Status

The report integrates the three members' supplied texts with saved training artifacts and the analysis package supplied on 23 September 2026. It distinguishes implementation, member-supplied execution artifacts and verification during report integration. Prediction metrics, identities and timing summaries were checked from exported records; checkpoint byte hashes were compared with the supplied weights. No training, checkpoint deserialization, independent model execution, new GPU benchmark or new test-suite execution was performed during report integration. Hash and record consistency are not substitutes for independent execution.

Item	Evidence currently available	Remaining qualification
Data/EDA and split	Academic dataset citation; repository EDA figures; recorded split-validation command/output; partition sizes and measured mean/std outputs	Exact linkage of copied EDA figures to a pinned execution, sampling provenance, and raw-content quality/duplicate auditing remain incomplete
Dataset/DataLoader and Linear/MLP	Implemented dataset/loaders and Linear/MLP modules; runtime batch-contract example; run histories; recorded R03/R04 training/evaluation outputs	Baseline termination logs and complete clean environment/source restoration remain outstanding
Train/validation and artifacts	Saved predictions plus analysis-notebook command and successful output for all four runs	Notebook revision pinning and clean-session execution need review; integration did not rerun weights
Quantitative results and curves	Four full-budget-configured runs, distinct from short-budget Optuna exploration	Linear short-budget search completed; MLP search completion and some archive run classifications remain unverified
Efficiency	Supplied forward-only T4 timings and parameter counts for all four runs; recorded epoch-duration sums	Zero-input batch-64 timing, not end-to-end or batch-one latency; no repeated-session uncertainty
Qualitative/error analysis	Four confusion matrices and highest-confidence error galleries; exported predictions checked	Galleries are deliberately selected, not random; correct/difficult-correct examples remain optional additions

Table 0.1: Draft evidence status and remaining qualifications.

1 Part 1 — Problem and Data Description

1.1 Problem Formulation

The task is to assign one of ten clothing-category labels to each grayscale image. A sample is an image-label pair (x, y) , where the preprocessed image has shape $[1, 28, 28]$ and y is an integer in 0–9. A model maps a batch to ten logits per image; the predicted class is the index of the largest logit. The benchmark examines how an affine baseline and a nonlinear fully connected network behave under a shared data and evaluation pipeline.

The central draft question is whether the MLP’s additional capacity improves validation performance relative to Linear, and what limitations prevent a stronger causal or efficiency conclusion. The small, standardized input makes controlled experimentation practical, but findings should not be generalized directly to high-resolution photographs or deployment conditions. Architectural inductive bias and dataset bias are discussed separately.

1.2 Dataset Identity and Scope

Fashion-MNIST is the primary benchmark dataset used in this study. It was introduced by Xiao, Rasul, and Vollgraf as a direct drop-in replacement for the original MNIST digit dataset^[1].

- **Primary citation:** Xiao, H., Rasul, K., and Vollgraf, R. (2017). *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*.
- **Dataset license:** Released under the MIT License by Zalando Research. This is legally distinct from the BSD-3-Clause license governing the `torchvision` wrapper library.

1.2.1 Collection, Labeling, and Processing Methodology

The images were collected directly from Zalando’s online catalog across ten clothing and footwear categories. The original assets were high-resolution product photographs taken by professional photographers against clean, neutral backgrounds. Zalando Research processed and standardized the images to match the MNIST format through the following steps:

- **Grayscale conversion:** Catalog images were converted to single-channel 8-bit grayscale.
- **Lanczos resampling:** The longer dimension of each image was downsampled to 28

pixels using Lanczos filtering.

- **Center-of-mass alignment:** The garment was positioned onto a 28×28 bounding canvas by aligning the center of mass of the item with the geometric center of the canvas.
- **Pixel inversion:** Colors were inverted so that 0 represents the black background and intensity values from 1 to 255 represent the foreground clothing material.
- **Class labeling:** The dataset is partitioned into ten mutually exclusive classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

1.2.2 Acquisition, Formats, and Measured Storage Size

The raw files were acquired from the canonical [Kaggle Fashion-MNIST distribution mirror](#), [zalando-research/fashionmnist](#), and staged locally within the project under `data/FashionMNIST/raw/`. The raw data is stored in the canonical binary IDX format: big-endian ubyte arrays with standard magic-number headers.

- **train-images-idx3-ubyte:** 47,040,016 bytes uncompressed (~ 47.04 MB or 44.86 MiB); 26,577,463 bytes packed gzip (~ 26.58 MB or 25.35 MiB). Header: 16 bytes; data: $60,000 \times 28 \times 28$ bytes.
- **train-labels-idx1-ubyte:** 60,008 bytes uncompressed (~ 60.01 kB or 58.60 KiB); 29,478 bytes packed gzip (~ 29.48 kB or 28.79 KiB). Header: 8 bytes; data: 60,000 label bytes.
- **t10k-images-idx3-ubyte:** 7,840,016 bytes uncompressed (~ 7.84 MB or 7.48 MiB); 4,447,700 bytes packed gzip (~ 4.45 MB or 4.24 MiB). Header: 16 bytes; data: $10,000 \times 28 \times 28$ bytes.
- **t10k-labels-idx1-ubyte:** 10,008 bytes uncompressed (~ 10.01 kB or 9.77 KiB); 5,107 bytes packed gzip (~ 5.11 kB or 4.99 KiB). Header: 8 bytes; data: 10,000 label bytes.
- **Total raw binary footprint:** 54,950,048 bytes (~ 54.95 MB or 52.40 MiB) uncompressed; 31,059,748 bytes (~ 31.06 MB or 29.62 MiB) packed gzip.

The companion tabular exports are `fashion-mnist_train.csv`, at 133,047,193 bytes (~ 133.05 MB or 126.88 MiB; 35,193,870 bytes packed), and `fashion-mnist_test.csv`, at 22,176,691 bytes (~ 22.18 MB or 21.15 MiB; 5,860,240 bytes packed).

Fashion-MNIST: reported dataset file sizes

Values transcribed from Section 1.2.2 of the supplied report.

File	Uncompressed bytes	Packed bytes
train-images-idx3-ubyte	47,040,016	26,577,463
train-labels-idx1-ubyte	60,008	29,478
t10k-images-idx3-ubyte	7,840,016	4,447,700
t10k-labels-idx1-ubyte	10,008	5,107
fashion-mnist_train.csv	133,047,193	35,193,870
fashion-mnist_test.csv	22,176,691	5,860,240

Original measurement image was not supplied; values have not been independently remeasured.

Packed values are reproduced as reported; no new dataset files were downloaded or measured.

Figure 1.1: Dataset file sizes reported in Section 1.2.2; values were not independently remeasured.

1.3 Exploratory Data Analysis

The supplied EDA covers class distributions, sample counts, image dimensions and representative examples. The published class-distribution figure shows 5,000 training, 1,000 validation and 1,000 test examples for each class. Accordingly, the displayed within-partition maximum/minimum class-count ratio is 1.0. This reduces class-frequency imbalance as an explanation of differences, but does not establish equally difficult classes or absence of content/label bias.

Class ID	Label	Train	Validation	Test
0	T-shirt/top	5,000	1,000	1,000
1	Trouser	5,000	1,000	1,000
2	Pullover	5,000	1,000	1,000
3	Dress	5,000	1,000	1,000
4	Coat	5,000	1,000	1,000
5	Sandal	5,000	1,000	1,000
6	Shirt	5,000	1,000	1,000
7	Sneaker	5,000	1,000	1,000
8	Bag	5,000	1,000	1,000
9	Ankle boot	5,000	1,000	1,000

Table 1.1: Class counts by dataset partition.

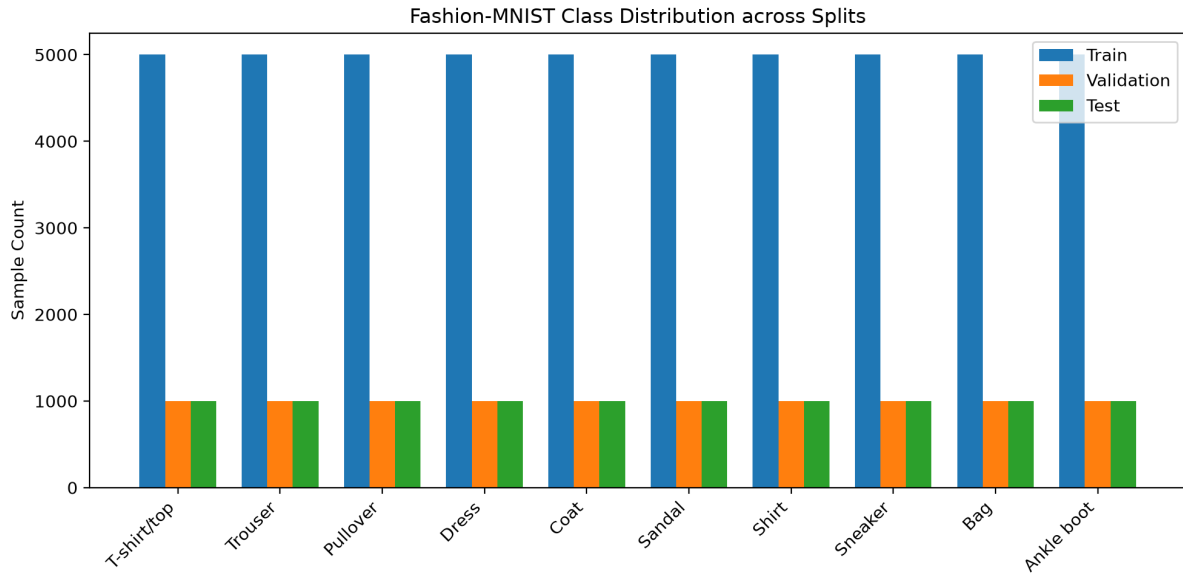


Figure 1.2: Repository EDA: class counts by partition.

Counts in Table 1.1 are transcribed from the supplied repository figure. Training notebook N-TRAIN calls `prepare_data(linear_config)` and the `generate_eda` routine, then displays `class_distribution.png`. Its output records partition sizes of 50,000/10,000/10,000. Exact linkage of the copied figure to that execution and its source revision remains for owner confirmation; this integration did not recompute raw-label counts.

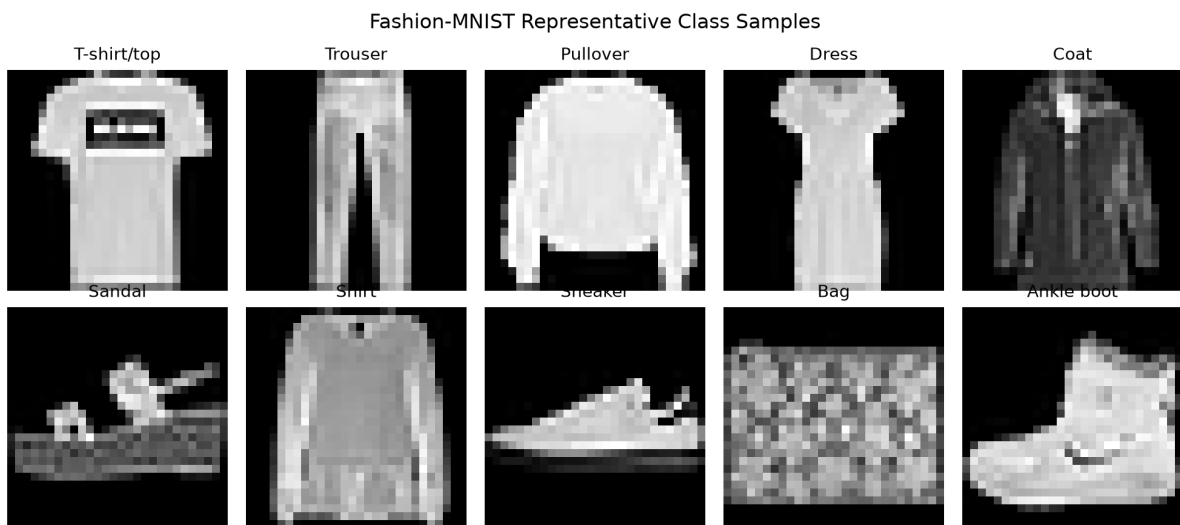


Figure 1.3: Repository representative Fashion-MNIST samples.

The submitted descriptions do not establish a full scan for unreadable images,

invalid labels, constant images or duplicate image content. Index-disjointness checks are not equivalent to content-duplicate checks. A pixel range measured on one image must not be described as a dataset-wide measured range. These limitations should remain explicit unless the corresponding audit output is supplied.

1.4 Split and Leakage Controls

Partition	Original source	Count	Use
Train	Official training set	50,000	Gradient updates and normalization fitting
Validation	Official training set	10,000	Configuration and checkpoint selection
Test	Official test set	10,000	Reserved for the final evaluation protocol

Table 1.2: Dataset split and intended use.

The saved stratified split uses seed 36. The manifest is stored at `configs/a1/splits/fashion_mnist_split.json` and is available in the [versioned GitHub source tree](#). Validators enforce train/validation disjointness, duplicate-free indices, the official 60k/10k partitioning, and preservation of the official test partition. A run seed changes model and training randomness only; it does not alter the membership of the split.

Sample identities are source-qualified, so equal numeric indices from different original sources are not treated as the same sample. For example, `official_train:42` and `official_test:42` are distinct samples. Hash agreement confirms artifact provenance and consistency, but it does not independently prove that every membership check has been executed and passed.

The split-validation evidence was generated under code revision 3889a31. The validation command was:

Listing 1: Split-validation command recorded for the project.

```
.\.venv\Scripts\python.exe -c "  
import sys; sys.path.insert(0, 'src')  
from dlbench.common.config import load_config  
from dlbench.a1.data.split import load_split, validate_split_against_config  
cfg = load_config('configs/a1/models/linear.py')  
manifest = load_split('configs/a1/splits/fashion_mnist_split.json')  
validate_split_against_config(manifest, cfg['data'])
```

```
print('dataset=', manifest.dataset)
print('split_seed=', manifest.split_seed)
print('train_size=', len(manifest.train_indices))
print('validation_size=', len(manifest.validation_indices))
print('test_size=', len(manifest.test_indices))
print('train_head=', manifest.train_indices[:5])
print('validation_head=', manifest.validation_indices[:5])
print('test_head=', manifest.test_indices[:5])
print('train_val_disjoint=',
      set(manifest.train_indices).isdisjoint(manifest.validation_indices))
print('train_val_partition_ok=',
      set(manifest.train_indices) | set(manifest.validation_indices)
      == set(range(60000)))
print('test_partition_ok=',
      set(manifest.test_indices) == set(range(10000)))
"
```

The resulting validation output was:

Listing 2: Recorded split-validation output.

```
dataset= fashion_mnist
split_seed= 36
train_size= 50000
validation_size= 10000
test_size= 10000
train_head= [0, 1, 2, 6, 7]
validation_head= [3, 4, 5, 8, 9]
test_head= [0, 1, 2, 3, 4]
train_val_disjoint= True
train_val_partition_ok= True
test_partition_ok= True
```

This document reports the preserved split protocol, split validation, and provenance, but it does not report an official test-set score and does not claim exhaustive auditing of all historical test access.

1.5 Preprocessing and Augmentation

The selected run configs agree on mean 0.28585961086061534 and standard deviation 0.352790375749525. Thiên reports measuring these from the unaugmented 50,000-image training partition after scaling pixels to $[0, 1]$. The supplied N-TRAIN notebook prints these same values as `measured_mean` and `measured_std` from `prepare_data` for both adjusted runs. This adds recorded execution output to the config evidence; the values were not independently recomputed during report assembly, and the precise historical source revision remains part of the reproduction record.

Stage	Train	Validation/test
Augmentation	Random crop to 28×28 , padding 2, constant fill 0	None
Conversion	Tensor conversion and scaling to $[0, 1]$	Same
Normalization	$(x - \text{mean})/\text{std}$ using training statistics	Same training statistics
Output	<code>float32[B, 1, 28, 28]</code>	Same

Table 1.3: Preprocessing and augmentation pipeline.

Augmentation precedes tensor conversion and normalization. Normalized inputs need not remain in $[0, 1]$. The training-only transformation helps separate evaluation stability from stochastic training variation.

1.6 Dataset and DataLoader Design

`FashionMNISTSubset.__getitem__` maps a local subset position to its original image/label and returns a stable ID of the form `fashion_mnist:official_train:<index>` or `fashion_mnist:official_test:<index>`. `collate_samples` stacks images/labels and retains IDs in matching order. The engine consumes the resulting `Batch`; the model itself receives only `batch['images']`.

Training uses shuffling with a dedicated run-seeded generator. Validation/test do not shuffle or drop the final batch. Selected-run batch size is 64. `seed_worker` derives Python/NumPy worker seeds from `torch.initial_seed() % 2**32`. Non-shuffled loading alone does not guarantee determinism; transforms, kernels and RNG settings also matter.

The active `DataLoader` contract was verified in the project runtime environment using a diagnostic override of batch size 4. This small batch was used only to inspect one returned batch; it is not the training configuration of R01–R04, which all use batch size 64. The diagnostic loader used 0 worker processes, `pin_memory=False`, and `drop_last=False`. Its first batch produced image tensors of shape $(4, 1, 28, 28)$ and dtype `torch.float32`, while labels had shape $(4,)$ and dtype `torch.int64`. Sample IDs were preserved in order as `fashion_mnist:official_train:47557` and `fashion_mnist:official_train:4416`. This confirms that the batch contract returned stacked image tensors, aligned labels, and stable source-qualified sample identities in the inspected runtime environment.

Listing 3: DataLoader contract-validation command.

```
.\.venv\Scripts\python.exe -c "import sys
sys.path.insert(0, 'src')
from dlbench.common.config import load_config
from dlbench.a1.data.dataset import prepare_data
from dlbench.a1.data.loaders import build_data loaders
cfg = load_config('configs/a1/models/linear.py')
cfg['training']['batch_size'] = 4
cfg['run']['seed'] = 69420
cfg['data']['download'] = False
prepare_data(cfg)
loaders = build_data loaders(cfg, smoke=False)
batch = next(iter(loaders.train))
print('images_shape=', tuple(batch['images'].shape))
print('images_dtype=', batch['images'].dtype)
print('labels_shape=', tuple(batch['labels'].shape))
print('labels_dtype=', batch['labels'].dtype)
print('sample_ids_head=', batch['sample_ids'][:2])
print('batch_size=', loaders.train.batch_size)
print('num_workers=', loaders.train.num_workers)
print('pin_memory=', loaders.train.pin_memory)
print('drop_last=', loaders.train.drop_last)"
```

Listing 4: Recorded DataLoader validation output.

```
images_shape= (4, 1, 28, 28)
images_dtype= torch.float32
labels_shape= (4,)
labels_dtype= torch.int64
sample_ids_head= ['fashion_mnist:official_train:47557',
                  'fashion_mnist:official_train:4416']
batch_size= 4
num_workers= 0
pin_memory= False
drop_last= False
```

2 Part 2 — Methodology

2.1 Shared Pipeline and Implementation Boundaries

```
Literal model config + shared protocol → load/validate → CLI → trainer
Raw images + saved split → per-split transforms → Dataset/DataLoader
train batch → model logits → cross-entropy → backward/optimizer
validation batch → model.eval + inference_mode → aggregate metrics
epoch results → history → scheduler → best/last checkpoints
Saved artifacts → learning curves/summaries; predictions and timing separately
```

The project uses PyTorch layers, autograd, optimizers and loaders^[3], torchvision data/transforms and library metric operations through project wrappers. The group implements the configuration policy, wrappers/split handling, architecture assembly, orchestration, checkpoint selection, artifact rules and analysis integration. It does not claim to have implemented autograd or framework primitives from scratch.

2.2 Interfaces and Representations

Batch contains `float32` images $[B, 1, 28, 28]$, `int64` labels $[B]$ and stable sample IDs. Both models flatten internally to $[B, 784]$ and return raw logits $[B, 10]$. `EpochMetrics` contains loss, accuracy, macro-F1 and sample count. `EvaluationResult` couples aggregate metrics with `Predictions`; `FitResult` identifies the run directory, selected checkpoint and history. Type annotations describe contracts but do not enforce all runtime shapes by themselves.

2.3 Linear Classifier

The model applies `nn.Flatten(start_dim=1)` followed by `nn.Linear(784,10)` with bias. For flattened input x , logits are

$$z = Wx + b,$$

with W of shape $[10, 784]$ and b of shape $[10]$. The prediction is $\arg \max(z)$. There is no hidden layer and no softmax before cross-entropy. The architecture-derived parameter count,

$$784 \times 10 + 10 = 7,850,$$

agrees with the count in both supplied Linear benchmark records. This integration did not itself reconstruct the models.

Pairwise decision boundaries are affine hyperplanes. The baseline is simple and inexpensive in parameter count but cannot represent general nonlinear pixel interactions.

Initialization follows the PyTorch layer defaults unless overridden; owner confirmation against the actual training revision remains necessary.

2.4 Multilayer Perceptron

The four selected configurations use the following MLP topology where applicable:

Layer	Output shape	Operation
Flatten	$[B, 784]$	Preserve batch dimension
Hidden 1	$[B, 256]$	Linear + ReLU + Dropout
Hidden 2	$[B, 128]$	Linear + ReLU + Dropout
Classifier	$[B, 10]$	Linear, raw logits

Table 2.1: MLP architecture used by the selected runs.

Dropout^[5] is 0.1 in the baseline and 0.2 in the selected adjusted MLP. The architecture-derived parameter count is

$$(784 + 1) \times 256 + (256 + 1) \times 128 + (128 + 1) \times 10 = 235,146,$$

approximately 30 times the Linear count. ReLU^[4] and dropout add no trainable parameters. Both supplied MLP benchmark records report the same count; their checkpoint hashes match the corresponding archived weights.

The forward computation is

$$h_1 = \text{ReLU}(W_1x + b_1), \quad h_2 = \text{ReLU}(W_2h_1 + b_2), \quad z = W_3h_2 + b_3,$$

with dropout active after the hidden activations during training. Without nonlinear activations, successive affine layers collapse to one affine map. Nonlinearity enables more expressive decision regions, but no particular hidden unit is claimed to detect a sleeve or sole without feature analysis. Flattening preserves values but provides no explicit locality/translation-sharing prior. Capacity and regularization can affect generalization in addition to nonlinearity.

2.5 Loss and Metrics

For a true class y and logits z , cross-entropy is

$$-z_y + \log \left(\sum_j e^{z_j} \right)$$

[6]. Predictions use $\arg \max$; softmax is used only for optional probability export. Epoch loss is the sample-weighted average of batch losses, so a smaller final batch is not overweighted.

Accuracy is correct predictions divided by sample count. For each class k ,

$$F1_k = \frac{2TP_k}{2TP_k + FP_k + FN_k},$$

with undefined cases assigned zero under the metric policy. Macro-F1 averages these scores over all ten labels^[7]. Predictions are accumulated over the full split rather than averaging batch F1 values. Class order remains fixed at 0–9.

2.6 Training and Validation

`train_one_epoch` sets train mode, transfers tensors to the selected device, clears gradients, computes logits/loss, runs backward and steps the optimizer. `evaluate_epoch` sets evaluation mode and uses `torch.inference_mode()` without optimizer updates. It returns aligned sample IDs, labels, predictions and probabilities in memory. This return value is not evidence that a prediction file was persisted.

After each epoch, the trainer appends history, steps the scheduler, applies checkpoint selection, updates early-stopping counters and saves resumable state at the configured frequency or termination. The recorded learning rate precedes that epoch's scheduler step. For `ReduceLROnPlateau`, the monitored quantity follows the configured mode. For cosine scheduling^[10], the scheduler advances each epoch. Best-checkpoint selection and early stopping use distinct comparisons, so the final stopping epoch need not equal the best epoch plus patience.

2.7 Actual Selected-Run Configurations

Table 2.2 uses saved run configs, not the latest repository defaults. R01–R04 are defined in Section 3.2. All use batch size 64, run seed 69420 and the shared preprocessing above.

Setting	R01 Linear baseline	R02 MLP baseline	R03 Linear adjusted	R04 MLP adjusted
Optimizer	Adam	Adam	AdamW	AdamW
Learning rate	0.001	0.001	0.0003	0.0005
Weight decay	0	0	0.001	0.0001
Epoch cap	100	100	200	200
Scheduler	Cosine	Cosine	ReduceLROnPlateau	Cosine
Scheduler parameters	$T_{\max} = 100, \eta_{\min} = 10^{-5}$	Same	mode=max, factor=0.5, patience=5, min lr = 10^{-5}	$T_{\max} = 200, \eta_{\min} = 10^{-5}$
Early-stop monitor	macro-F1	macro-F1	macro-F1	macro-F1
Patience / min_delta	15 / 0.001	15 / 0.001	20 / 0.0005	15 / 0.0005
Save frequency	1000	1000	10	10
Hidden dropout	N/A	0.1	N/A	0.2

Table 2.2: Actual selected-run configurations.

The configured `tuning_trials_per_model=1` does not describe the full observed experiment history: the notebook separately invokes Optuna^[11] with ten requested trials per model (Section 3.9.1). Momentum fields present in Adam/AdamW configs are not evidence those optimizers used SGD-style momentum arguments. Several factors changed together, so the adjusted runs are not single-factor ablations.

Here, *full-budget runs* means training with the 100/200-epoch caps and configured early stopping, not necessarily executing every allowed epoch. R01–R04 are not presented as full-budget retraining of the Optuna-selected parameters.

2.8 Checkpoints, Configuration and Provenance

Checkpoint selection maximizes validation macro-F1, breaking exact ties by lower validation loss and then earlier epoch. Non-finite candidate loss/F1 values are rejected. `best.pt` is the selected checkpoint; `last.pt` supports continuation with optimizer/scheduler/RNG state and trainer bookkeeping. The trainer may include optimizer/RNG fields in best checkpoints too; the distinction is purpose and required resumable fields, not strict mutual exclusion of contents.

Checkpoint writing uses a temporary file in the target directory and `os.replace`, avoiding direct partial overwrite of the existing target during serialization. This is not a general power-loss durability guarantee. The inspected loader deserializes trusted checkpoint files and checks dictionary/required-key presence; it does not validate every value/type or schema version. Untrusted pickle-based checkpoints must not be loaded.

Configuration files contain literal `CONFIG` dictionaries parsed as data, not imported executable programs. Model-specific files cannot override common protocol sections. Strict validation rejects unresolved main-run decisions but does not prove hardware availability or successful training.

Run metadata records config snapshots, code revision/dirty status, split identity, normalization and environment. Source/split hashes use raw bytes; statistics use compact key-sorted JSON. Metadata writing is not an all-or-nothing multi-file transaction. History validates its fixed columns, finite values and increasing epochs; saved-run analysis additionally requires contiguous epochs from zero. Validation/test metric files are separate; explicit overwrite backs up old bytes before replacement under a single-writer assumption.

Resume checks relevant configuration sections and provenance, restores state and reconciles history. These checks do not capture every possible code/hardware difference. Exact continuation is not claimed solely from RNG restoration or hash agreement.

2.9 Fairness and Test Policy

The reported runs share the recorded split and normalization identities, evaluation metrics and run seed. Baseline/adjusted changes and differing executed epochs are disclosed. Validation drives checkpoint selection and exploratory configuration changes. CLI test evaluation requires explicit acknowledgement and cannot run in smoke mode. This is a workflow guard, not enforcement of one-time test access, nor proof that all prior runs avoided test-based decisions. No test results or across-seed mean/std are presented.

2.10 Randomness and Environment

The project seeds Python, NumPy and PyTorch. It calls `torch.use_deterministic_algorithms` according to configuration. Worker generators and RNG snapshots support reproducibility; cross-device/version bitwise equality is not guaranteed.

All four selected training `environment.json` files record Linux x86_64, Tesla T4, Python 3.13.15, PyTorch 2.11.0+cu128, torchvision 0.26.0+cu128, NumPy 2.1.3, CUDA build 12.8 and deterministic algorithms enabled. The training artifacts do not record the effective training precision, TF32 flags, RAM/VRAM capacity or cuDNN version, so their historical values are unavailable for this report. The later forward-only benchmark records report FP32, highest float32 matmul precision, CUDA matmul TF32 disabled, cuDNN TF32 allowed, one CPU thread and deterministic algorithms disabled. Those benchmark settings describe the later measurement only; they cannot be assigned retrospectively to training. Versions and settings above are artifact-reported, not independently inspected on the original machine. Future runs should log the missing settings when they execute.

2.11 Efficiency and Analysis Method

Saved-run analysis reads configuration, metadata, metrics and history, validates identities and consistency, and generates learning curves without loading weights. The later supplied package also contains exported validation probabilities/predictions, confusion counts, error galleries and per-run benchmark records. Prediction export is a separate analysis artifact; its presence does not imply the trainer automatically persisted it.

The inference tool implements forward-only timing with evaluation/inference mode, CUDA synchronization, warm-up and repetitions. Supplied measurements use a Tesla T4, FP32, batch 64, 20 warm-ups and 100 timed batches with preallocated zero inputs of shape [64, 1, 28, 28]. Reported median times were checked against the 100 saved timing samples. Amortized time is median batch time divided by 64; throughput is 64,000 divided by median batch milliseconds. These derived quantities are not independent measurements. Data loading, preprocessing, host-to-device transfer and checkpoint I/O are excluded. Amortized batch time per image is not batch-one latency, and no repeated-session variability estimate is available.

The time values in Section 3.4 sum recorded `epoch_seconds`. They are neither inference time nor complete fit wall time. Based on the inspected trainer, the timer surrounds train/validation and precedes subsequent history/checkpoint I/O. Exact historical boundaries require the recorded source revision, especially for dirty runs.

3 Part 3 — Implementation Results and Draft Discussion

3.1 Verification Record

The supplied N-TRAIN notebook records `fit(..., smoke=False)` and `evaluate_checkpoint(..., split="validation", smoke=False)` for R03/R04, prints their exact best-checkpoint paths and reproduces the metrics in Section 3.4 for 10,000 samples. Its terminal progress shows patience counters 20/20 for Linear and 15/15 for MLP, supporting early-stopping termination for those adjusted runs. Baseline termination logs are not supplied in this notebook. N-ANALYSIS records successful `postprocess_run.py -trust-checkpoint -benchmark` output for all four selected run IDs. These are saved member execution outputs, not newly executed checks by the report integrator.

The member-supplied integration account records successful CPU smoke training/evaluation on 64 validation samples: Linear accuracy 0.59375, macro-F1 0.59877; MLP accuracy 0.59375, macro-F1 0.54230. These debug results are separate from the main-run results below.

The archived execution result reported for 21 September 2026 records 190 discovered tests: 187 passed and three skipped, with no failures or errors (18.153 seconds; exit code 0). The execution tested the working-tree implementation subsequently committed as `d9f2295291be698ade3693120f6a55f8189485ce`, rather than a clean checkout of that commit. Two skipped tests concern shared contracts covering all five architectures, including models not yet implemented for the Draft. The third is a stale skip on the split-overlap rejection regression test: overlap validation is implemented, but the skipped negative test remains a coverage gap. The dated log, revision metadata and skip explanations are packaged in the linked [a1-test-evidence.zip archive](#). The supplied archive was inspected during report review, and its file hashes matched the included SHA-256 manifest. No new test execution was performed as part of that review. These records support unit-test verification, not remote CI status or model-performance claims.

Checks of the supplied analysis on 23 September established the following: each run has 10,000 prediction rows and 10,000 unique IDs matching the saved validation index set; exported ground-truth mappings agree across the four runs; all probability vectors contain ten finite values in range and sum to one within 2.24×10^{-7} ; argmax predictions agree with the labels exported as predictions. Recalculated accuracy and macro-F1 agree with the saved metrics, and probability-derived cross-entropy differs by less than 2.4×10^{-9} . Confusion counts and error totals agree with the exports. The benchmark checkpoint

SHA-256 matches the corresponding archived `best.pt` for all four runs, and raw timing medians match the benchmark summaries. These checks do not independently validate raw image labels or prove which weights generated a prediction export without the generation log.

3.2 Selected Run Inventory

ID	Model/group	Full saved run ID
R01	Linear baseline	<code>a1_linear_main_seed69420_20260920-071742_9e5d8b</code>
R02	MLP baseline	<code>a1_mlp_main_seed69420_20260920-073429_497541</code>
R03	Linear adjusted	<code>a1_linear_main_seed69420_20260920-114825_3d2881</code>
R04	MLP adjusted	<code>a1_mlp_main_seed69420_20260920-120316_b22d28</code>

Table 3.1: Selected runs used in the draft comparison.

The selected R01–R04 run directories and checkpoints are available in the [R01–R04 run archive](#).

All four use seed 69420 and validation. R01/R02 record clean working trees at revision `54c4470a42da59423c97da152291a665dd1fde9b`. R03/R04 were executed from revision `17ca343595fbba82e88aea51cbf80d0c351dad39` with uncommitted configuration changes. The effective configuration saved inside each run is therefore the authoritative record of the settings summarized in Table 2.2. The selected set follows the supplied member report; it is not claimed to contain every trial or the globally best configurations in the archive.

3.2.1 Adjusted-Run Revision Qualification

R03/R04 ran with uncommitted changes to the model and protocol configurations. The supplied screenshots corroborate the main optimizer, learning-rate, regularization, early-stopping, scheduler and epoch-budget differences, but they are not a complete reconstructable patch. This limits exact source reconstruction. It does not change the saved metrics or configurations used in this report, and the adjusted runs are treated as multi-factor comparisons rather than single-factor ablations.

The configuration-diff evidence is available in the [dirty-tree evidence folder](#).

3.3 Learning Curves

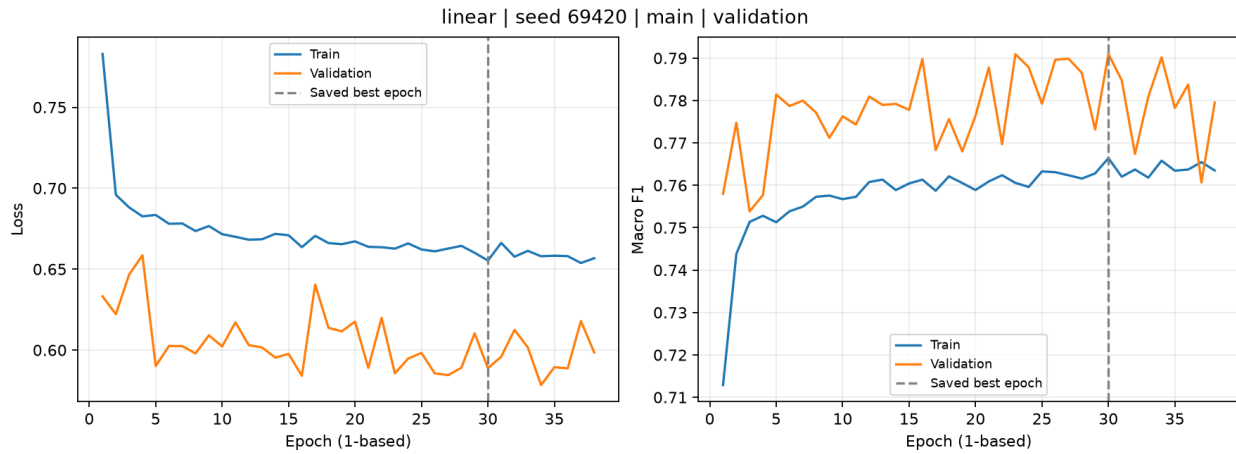


Figure 3.1: R01 Linear baseline training/validation curves. Seed 69420; 38 epochs. The dashed line marks saved best epoch 29, displayed as epoch 30.

Training loss decreases rapidly at first, while validation fluctuates. Training augmentation can contribute to validation outperforming train-mode metrics; this is not evidence of leakage by itself.

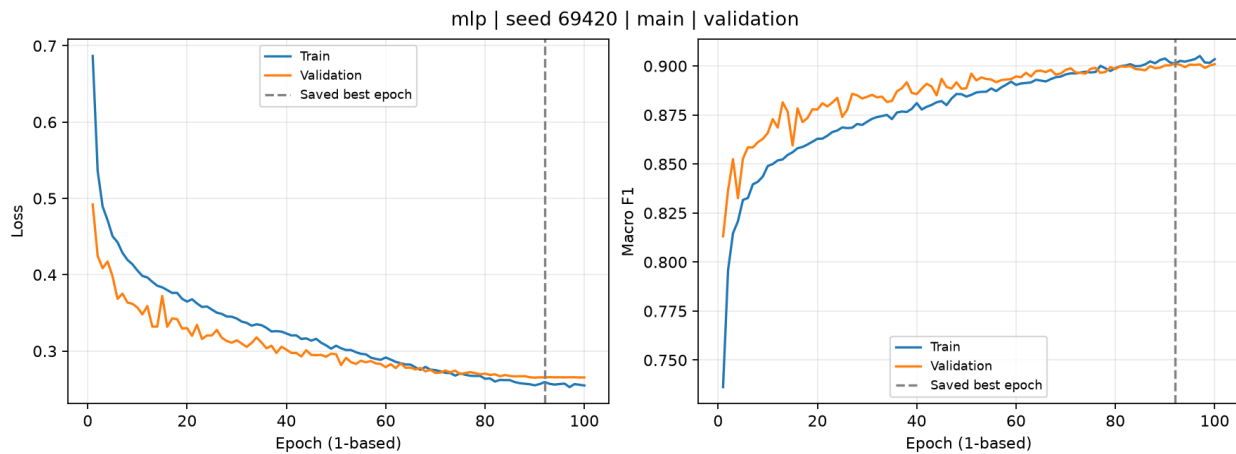


Figure 3.2: R02 MLP baseline training/validation curves. Seed 69420; 100 recorded epochs, best epoch 91, displayed as epoch 92.

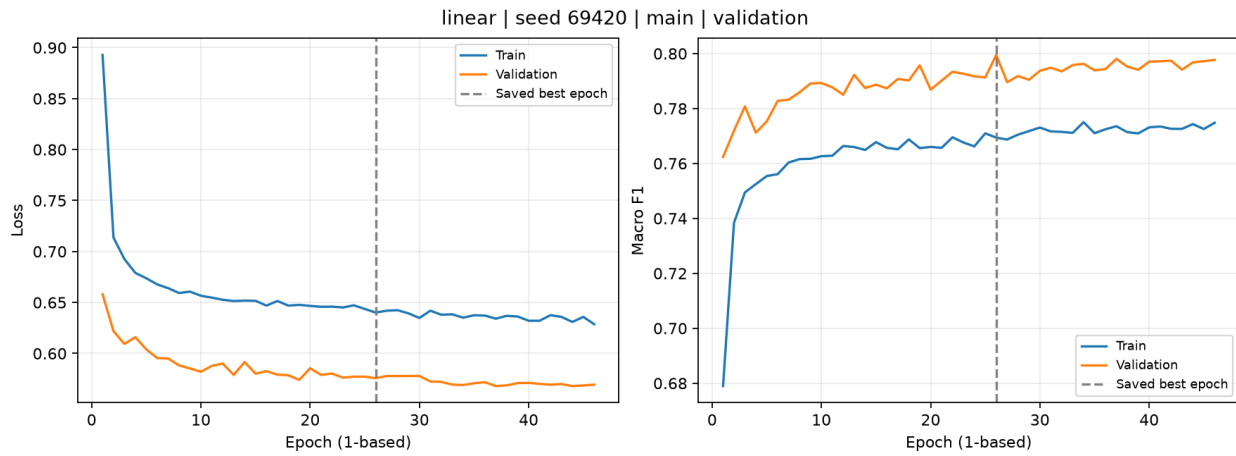


Figure 3.3: R03 adjusted Linear training/validation curves. Seed 69420; 46 recorded epochs, best epoch 25, displayed as epoch 26.

R03 is interpreted from its saved effective configuration; the dirty working tree limits exact source reconstruction.

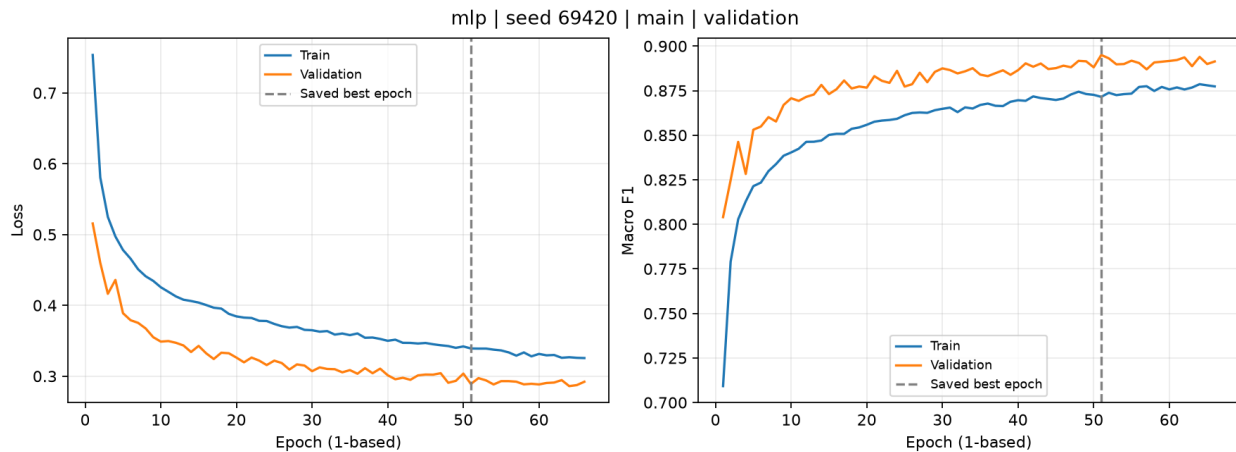


Figure 3.4: R04 adjusted MLP training/validation curves. Seed 69420; 66 recorded epochs, best epoch 50, displayed as epoch 51.

All four figures derive from saved histories, not newly evaluated checkpoints. Plot epochs are one-based; stored indices are zero-based.

3.4 Quantitative Results

Run	Val. loss	Accuracy	Macro-F1	Epochs	Best	Time (s)
R01	0.588840	79.16%	0.790977	38	29	751.26
R02	0.265457	90.15%	0.901141	100	91	2031.17
R03	0.575779	80.09%	0.799487	46	25	876.25
R04	0.288985	89.51%	0.895012	66	50	1302.69

Table 3.2: Selected validation results. Best epoch is zero-based; time is the sum of recorded epoch durations.

Metrics are the saved validation records, associated with the selected checkpoint epoch, and agree with recalculation from the later exports of 10,000 predictions per run. Epoch counts and durations were read from CSV histories. No standard deviation is calculated because these are not repeated seeds of a fixed configuration. N-TRAIN supplies patience-exhaustion output for R03/R04; baseline stop reasons still require their corresponding logs. Progress-bar counters are not substituted for the epoch counts in the saved histories.

3.4.1 Forward-Only Inference Measurements

Run	Parameters	Batch time (ms)	ms/image	Images/s
R01	7,850	0.068305	0.001067	936,974
R02	235,146	0.177286	0.002770	360,999
R03	7,850	0.070458	0.001101	908,336
R04	235,146	0.200205	0.003128	319,672

Table 3.3: Supplied forward-only inference microbenchmark.

All rows use the supplied batch-64, zero-input, FP32 T4 protocol. This is a forward-only microbenchmark, not an application throughput measurement. Values are rounded from each `benchmark.json`; near-rounding-boundary differences do not indicate different measurements. Training epoch sums and inference batch timings measure different scopes and must not be compared as interchangeable costs.

3.5 Qualitative Results

The analysis package provides ten highest-confidence mistakes for each run. Figure 3.5 shows the baseline MLP gallery. Each panel includes a true label, predicted label and rounded predicted-class probability. The gallery is selected to expose confident failures, not

estimate their prevalence. A displayed probability of 1.000 is rounded and is not proof of calibrated certainty. Sample identities remain in the prediction/error records; the original individual-image filenames also contain indices.

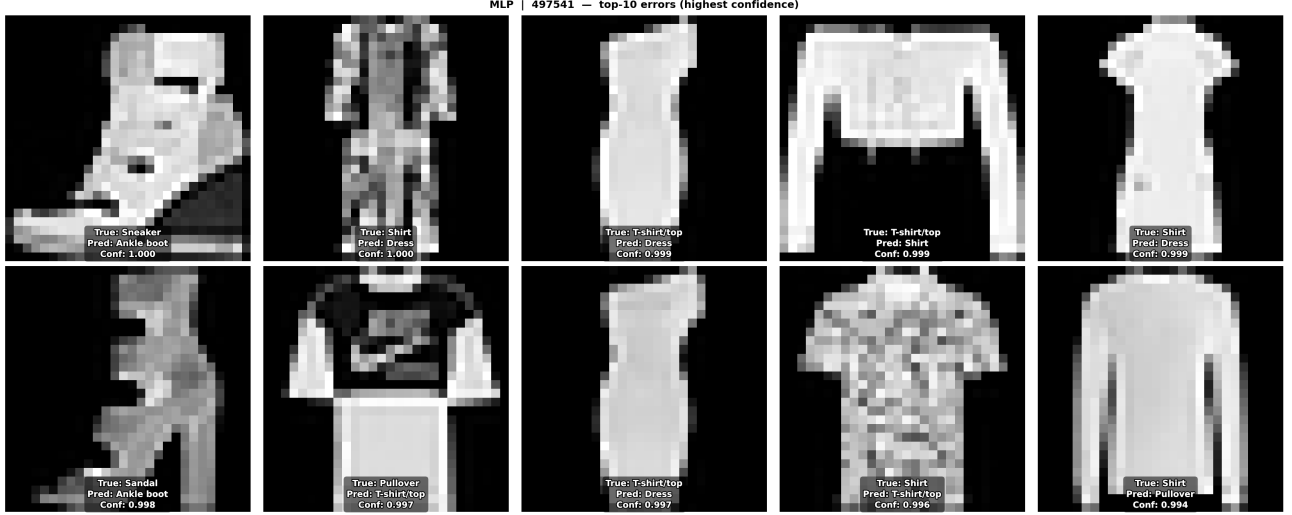


Figure 3.5: R02 baseline MLP highest-confidence validation mistakes.

These are model-error examples, unlike the input-only EDA samples in Figure 1.3. The selection excludes correct predictions and low-confidence mistakes. Original image-to-ID correspondence was not independently reconstructed during integration. Companion galleries for R01, R03 and R04 are provided in Appendix A. Correct and difficult-but-correct examples have not been supplied; they would broaden the qualitative discussion but are not represented here by fabricated examples. Confidence alone does not establish why the model failed or whether the dataset label is wrong.

3.6 Error Analysis and Representation

Run	Incorrect	Error rate	Three most frequent off-diagonal confusions
R01	2,084 / 10,000	20.84%	Coat → Pullover: 273; Pullover → Shirt: 215; Shirt → T-shirt/top: 171
R02	985 / 10,000	9.85%	Shirt → T-shirt/top: 120; T-shirt/top → Shirt: 110; Pullover → Coat: 83
R03	1,991 / 10,000	19.91%	Coat → Pullover: 224; Shirt → Pullover: 168; Shirt → T-shirt/top: 158
R04	1,049 / 10,000	10.49%	T-shirt/top → Shirt: 131; Shirt → T-shirt/top: 108; Pullover → Coat: 95

Table 3.4: Validation error counts and most frequent confusions.

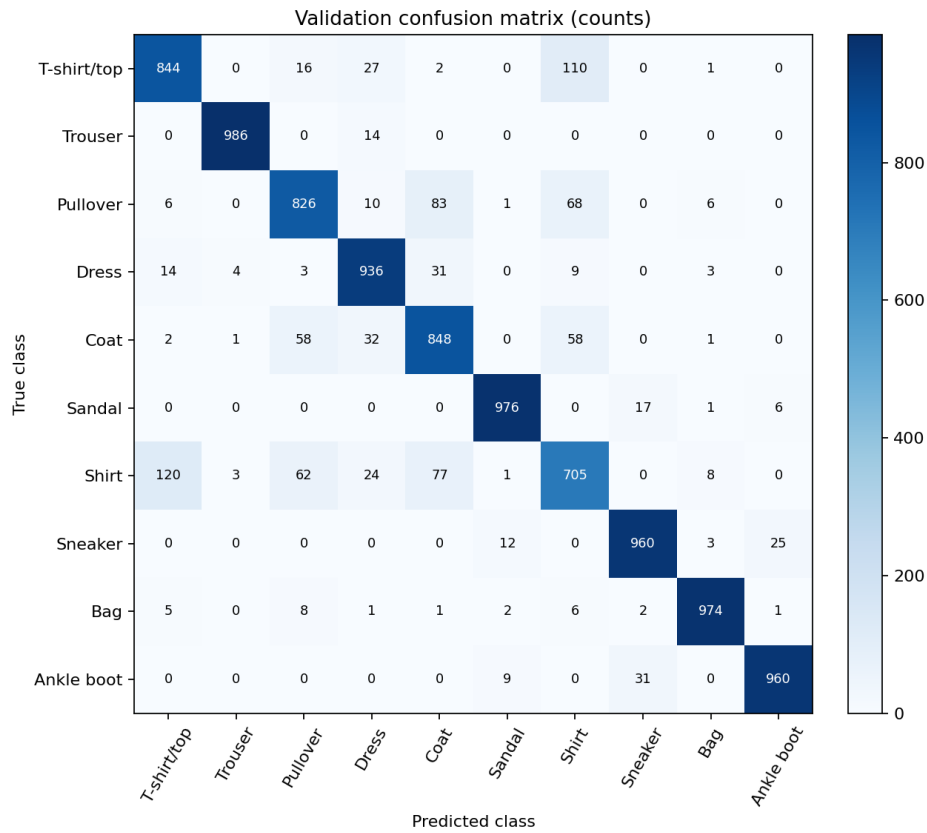


Figure 3.6: R02 baseline MLP validation confusion matrix. Rows are true classes and columns are predicted classes; entries are counts.

Each row contains 1,000 examples. All four matrices and underlying counts are included with the report; Appendix A displays the other three.

True class	R01 recall	R02 recall	R03 recall	R04 recall
T-shirt/top	78.4%	84.4%	77.0%	81.4%
Trouser	92.0%	98.6%	91.5%	98.1%
Pullover	65.9%	82.6%	69.7%	82.7%
Dress	84.4%	93.6%	85.7%	92.4%
Coat	53.3%	84.8%	62.8%	83.6%
Sandal	88.0%	97.6%	90.0%	95.7%
Shirt	52.1%	70.5%	47.9%	71.5%
Sneaker	94.1%	96.0%	90.9%	96.6%
Bag	91.6%	97.4%	92.2%	97.4%
Ankle boot	91.8%	96.0%	93.2%	95.7%

Table 3.5: Per-class validation recall.

The baseline MLP improves recall over baseline Linear for every class in this selected

pair. The largest increase is Coat, from 53.3% to 84.8%; Shirt remains the weakest MLP class at 70.5%. This class has 120 examples predicted as T-shirt/top, 77 as Coat and 62 as Pullover. Upper-body categories therefore remain a major source of confusion even with the nonlinear model. Similar silhouettes and the restricted grayscale resolution are plausible explanations, not causal findings established by these counts.

The adjusted Linear improves aggregate accuracy but reduces Shirt recall from 52.1% to 47.9%; Coat recall increases from 53.3% to 62.8%. Conversely, adjusted MLP raises Shirt recall from 70.5% to 71.5% while reducing overall accuracy. An aggregate improvement does not imply improvement for every class.

Balanced class counts address frequency imbalance only. They do not establish representative coverage of garment subtypes or acquisition conditions, and no demographic attributes were supplied for a demographic fairness analysis. Both models flatten the image, retaining pixels but lacking explicit spatial weight sharing. The MLP’s nonlinear representation is a plausible contributor to its stronger scores, but neither this comparison nor the error galleries identify specific learned features. Feature-level analysis would require additional evidence.

3.7 Comparison and Discussion

Within the baseline pair, MLP exceeds Linear by 10.99 percentage points in validation accuracy. Adjusting Linear raises accuracy by 0.93 points, while adjusting MLP lowers it by 0.64 points. The selected adjusted MLP therefore does not demonstrate an improvement over its baseline.

The results are consistent with benefits from nonlinear fully connected capacity, but do not isolate nonlinearity as the cause. Parameter count, regularization, optimization and training duration also differ. MLP has approximately 30 times as many parameters. In the supplied baseline microbenchmark, its median batch time is about 2.60 times Linear’s, in exchange for the observed 10.99-point validation-accuracy advantage. This is a descriptive tradeoff for these artifacts, not a universal hardware speed ratio. Differences between baseline and adjusted timings cannot be attributed to optimizer or dropout changes from these measurements alone; repeated controlled timing sessions would be needed.

MLP also has a larger recorded epoch-time sum in the baseline pair, but different executed epoch counts prevent interpreting that sum as pure per-step efficiency. The strongest selected validation result is baseline MLP; the adjusted configuration did not improve either aggregate accuracy or macro-F1. Class-level results qualify this ranking by exposing remaining Shirt errors and changes in per-class recall. No statistical significance

or five-model superiority claim is supported.

3.8 Implementation Issues and Corrections

The group separated split seed from run seeds, aligned the manifest path, added explicit metrics-overwrite backups, and separated lightweight CI from ML-dependent tests. Configuration tests use controlled fixtures rather than mutable project defaults. These changes support traceability but are not experimental improvements by themselves. The integration reference and run revisions are recorded separately to avoid claiming newer analysis code produced older training results.

3.9 Tuning Context and Additional Runs

The supplied archive contains 28 run directories: 24 have `metrics.json` and four do not. This inventory is broader than the four selected runs. Presence of a metrics file alone is not full validation of a run, and absence does not establish whether it failed, was interrupted or was only partially copied. Several additional configs change optimizer settings, hidden widths, dropout or budgets. They must not be pooled as repeated-seed uncertainty.

3.9.1 Short-Budget Optuna Exploration

The supplied training notebook N-TRAIN places Optuna exploration after the main training/evaluation sections. Its installation output records Optuna 5.0.0. Both objectives deep-copy a base config, suggest hyperparameters, call `fit`, evaluate the best checkpoint on validation and return validation macro-F1 to a study with `direction="maximize"`. The current code requests ten trials for each model, with a cap of 20 epochs and early-stopping patience 3. The function default of five trials is overridden by the explicit calls. Khoa reports earlier sessions of roughly 5–10 trials with 10–20-epoch budgets; the archive contains 10-epoch runs, but this notebook does not establish every earlier study’s configuration or trial mapping.

Model	Search space in the supplied code	Completion evidence
Linear	Learning rate 10^{-4} – 10^{-2} (log); weight decay 10^{-6} – 10^{-2} (log); batch size {64, 128, 256}	Ten trial progress outputs followed by a completion message and best parameters
MLP	Learning rate 10^{-5} – 10^{-3} (log); dropout 0.1–0.5; hidden width 1 {128, 256}, width 2 {64, 128}	Output stops during the first displayed trial; no completion message or best parameters

Table 3.6: Short-budget Optuna search spaces and available completion evidence.

The completed Linear search reports best validation macro-F1 0.7923 (rounded). Its parameters match archived run `a1_linear_main_seed69420_20260920-132409_9f26ea`: learning rate 0.0001696037388281409, weight decay 0.004205907412140831, batch size 64. That run records validation accuracy 0.7946, macro-F1 0.7923383862412255 and best epoch 12 (zero-based). These are short-budget exploratory results, excluded from the full-budget comparison and not evidence that longer retraining would improve R01–R04.

No full-budget training call using the selected Optuna parameters follows the search. Khoa also confirms these selected parameters have not been retrained. The cell intended to save `optimal_hyperparameters.json` has no saved success output; it requires both models' best-parameter variables. Consequently, successful export and completed MLP optimization are not claimed. The adjusted R03/R04 execution outputs precede the tuning section and identify the same checkpoints as the report; they are distinct from the Optuna-selected Linear configuration.

3.9.2 Tuning Qualifications and Archive Reconciliation

- Notebook prose says 10 epochs and promises pruning; executable code sets 20 epochs and contains no `trial.report`, `trial.should_prune` or `TrialPruned`. The implemented stopping mechanism is trainer early stopping, not demonstrated Optuna pruning.
- Exceptions are caught and returned as objective 0.0, obscuring failure status in the study. No persistent study storage, explicit sampler seed or saved trial-to-run mapping is provided. Training seed 69420 does not by itself specify the sampler's random state.
- Optimizer and scheduler settings are inherited from the base configuration. In the supplied MLP tuning config, cosine `T_max=200` remains unchanged despite a 20-epoch cap; the short run does not span the full cosine schedule. Linear's patience-3 early stopping can terminate a trial before its inherited patience-5 plateau scheduler reduces the learning rate.
- Best short-budget validation performance is a search result, not an estimate of full-budget performance or an independent test result. Config changes in memory are not fully described by the unchanged protocol ID or tuning-budget field.

Archive category	Count	Recorded epoch caps / qualification
Selected R01–R04 with metrics	4	100 or 200; full-budget-configured runs
Other runs with metrics	20	10 or 20; consistent with short-budget exploration
Runs without metrics	4	One cap 20, two cap 100, one cap 200; exact termination/copy status unresolved

Table 3.7: Archive reconciliation.

Thus, not every run outside the report can be classified as completed tuning.

3.10 Limitations

- Single selected run seed; no across-seed uncertainty estimate.
- Validation reuse for tuning can yield optimistic selection effects; no test generalization claim.
- Adjusted runs change multiple factors and were executed with uncommitted configuration changes. Their saved run configurations are authoritative, but exact source-tree reconstruction remains incomplete.
- Prediction exports were independently checked arithmetically; supplied notebooks now document adjusted-run training/evaluation and four-run postprocessing. No independent checkpoint execution or raw-label audit was performed during integration; complete revision-pinned clean-session reproduction remains outstanding.
- Supplied inference measurements cover zero-input forward passes only, with one timing session per run and no end-to-end or batch-one latency evidence.
- Benchmark deterministic algorithms are disabled, unlike the recorded training setting; execution phases must remain distinguished.
- Error galleries select the ten highest-confidence mistakes, not a representative sample; correct/difficult-correct examples and feature-level evidence are absent.
- Recorded split-validation output and a runtime DataLoader batch-contract example are available. Exact EDA figure provenance, raw-content quality/duplicate auditing and earlier tuning history remain incomplete.
- Linear short-budget Optuna optimization is documented; MLP study completion and best-parameter export are not. No Optuna-selected configuration has been demonstrated as full-budget retrained, and no seeded persistent study is supplied.

- Deterministic settings and matching hashes do not establish universal reproducibility or absence of leakage.

3.11 Draft Conclusion

The group has assembled a shared image-classification pipeline and supplied evidence for Linear and MLP execution, validation comparison, prediction-level error analysis and forward-only T4 timing. Recalculated exported-prediction metrics agree with the saved scores. Baseline MLP provides the highest selected validation accuracy at a larger parameter and measured forward-time cost, while the adjusted MLP does not improve its aggregate score. Upper-body garment confusions, especially Shirt, remain prominent. The report supports a bounded single-seed draft comparison, not a completed five-model benchmark. Immediate remaining work concerns verified public artifact links, owner review and accessible publication; additional architectures and multi-seed experiments belong to the Final stage.

3.12 Plan Toward Final

Owner	Work	Evidence required
Thiên	Custom CNN; complete EDA and visual error analysis	Model/config/tests, shared-engine runs and inspected figures
Khoa	LSTM/GRU; tuning inventory; evaluation/timing	Sequence definition, run logs, predictions and controlled measurements
Khải	Transformer; integration and provenance; report/Pages	Tokenization/position encoding, tests/runs, traceable publication artifacts
All	Full fair comparison and agreed seeds 69420/67/69	Compatible configs/splits, per-seed results, mean/std convention
All	Final report/slides/video and checkpoint access	Public links, clean reproduction and contribution review

Table 3.8: Plan toward the Final milestone.

Internal dates remain to be confirmed. The established Final milestone is 21 October 2026, subject to milestone-specific course announcements.

3.13 Reproducibility and Artifact Access

Artifact distribution uses the following group-owned access points:

- **Source repository:** [Deep Learning Benchmark Suite on GitHub](#)
- **Project website:** [Assignment website on GitHub Pages](#)

- **Draft artifacts and checkpoints:** [draft artifact archive on Drive](#)

The GitHub repository and project website were reachable without authentication during report integration. The Drive and Colab links should still be checked in a signed-out or private-browsing session before submission, especially the two shortcut-backed Drive folders, to confirm that readers outside the owning accounts can open the referenced runs, checkpoints, notebooks and test evidence.

All four selected analysis summaries report the following identities:

- split SHA-256: `e4a776305b2a1b5c392ba196d023de004f47124e8f831495b9733c48e72c25e6`;
- statistics SHA-256: `e47b863cb866cdc61613bb88d772d02f8aedc0a21fa56b79dee27b27d341381e`.

The analysis summaries retain input-artifact hashes. Each report evidence directory now also contains the supplied `benchmark.json`, `errors.json` and `predictions_validation.csv`. Curves, confusion matrices and error galleries are copied under `assets/`; raw dataset files and model weights are not copied. The older summaries are preserved unchanged as historical artifacts, including any now-outdated limitations about missing predictions; the 23 September package and this report supply the later evidence.

The original notebooks and their saved outputs are preserved as [N-TRAIN](#) and [N-ANALYSIS](#). Both show a clone of branch `a1-draft-test` followed by `pip install -e .`; N-TRAIN additionally shows `pip install optuna`. A branch name is not an immutable execution revision, and these commands do not alone restore all historical dependencies or dirty config changes.

N-TRAIN uses the Python API rather than the illustrative CLI commands in the earlier report. Its recorded calls include:

Listing 5: Recorded N-TRAIN API calls.

```
linear_config = load_config(linear_config_file)
mlp_config = load_config(mlp_config_file)
prepare_data(linear_config)
generate_eda(linear_config, output_dir=eda_output_dir)
linear_fit_result = fit(linear_config, smoke=False)
linear_eval_result = evaluate_checkpoint(
    linear_config, linear_fit_result.best_checkpoint,
    split="validation", smoke=False)
mlp_fit_result = fit(mlp_config, smoke=False)
mlp_eval_result = evaluate_checkpoint(
    mlp_config, mlp_fit_result.best_checkpoint,
    split="validation", smoke=False)
```

The variable paths and setup are retained in the full notebook; this excerpt is not a standalone script. N-ANALYSIS loops over the four IDs in Section 3.2 and invokes:

Listing 6: Recorded N-ANALYSIS postprocessing call.

```
cmd = [sys.executable, "scripts/postprocess_run.py",  
      "--run-dir", str(run_dir), "--output-dir", str(out_dir),  
      "--trust-checkpoint", "--benchmark"]  
result = subprocess.run(cmd, capture_output=True, text=True)
```

Its saved stdout reports validation-analysis success for all four IDs, and later cells generate the high-confidence error galleries. Original outputs are preserved, not represented as newly executed during integration.

Clean-session caveats remain: N-ANALYSIS references `project_root` before its definition, and its comparison-table code builds run paths without the `a1` component. The saved comparison output instead loads an existing file, so it does not verify that fresh-generation branch. Both notebooks contain setup code that removes an existing non-symlink output directory before linking Drive; these cells should not be rerun blindly on folders containing unsaved work. N-TRAIN also has the tuning documentation/status issues described earlier. Copies are evidence, not certified run-all reproduction notebooks.

3.14 Contributions and AI Disclosure

The contribution table records member-reported ownership and does not by itself certify group review or unaided authorship. The repository's [AI-assistance record](#) provides the project-level disclosure. Khải's text reports substantial Codex assistance with configuration/artifacts, tests, CLI/CI, analysis/benchmark code and report drafting. Commit authorship must not be represented as proof of unaided code authorship. Automated outputs, member-reported checks and final human review are distinct evidence types.

For this integration and its 23 September update, Codex combined member sections, checked saved artifacts and implementation details, recalculated prediction metrics and timing summaries, compared checkpoint byte hashes, added class-level error and efficiency discussion, and prepared the report and missing-items checklist. It did not retrain, deserialize checkpoint weights, independently execute model evaluation or measure new inference timings. Tạ Tuấn Khải is responsible for reviewing the integrated text, with Thiên validating data/Linear and Khoa validating trainer/MLP/results.

Throughout the development of Assignment 1, Nguyễn Anh Khoa utilized several AI assistants—including Gemini 3.8 Flash, Gemini 3.1 Pro (High), GitHub Copilot, and ChatGPT 5.6 Luna—primarily to accelerate testing, validate PyTorch implementation details, and clarify deep learning concepts. Early in the project, AI tools were used as a knowledge base to confirm the canonical execution sequence for a PyTorch training loop and to clarify the technical requirements for saving and resuming training states. This

ensured that the `engine.py` and `checkpoint.py` modules correctly handled optimizer, scheduler, and RNG states without disrupting training dynamics upon resumption.

The most significant portion of Khoa’s AI usage was dedicated to test-driven development and expanding test coverage. Using GitHub Copilot and Gemini, Khoa generated comprehensive unit tests for the MLP classifier, evaluation metrics, and the main trainer loop. These generated tests covered critical edge cases such as exact linear-layer dimensions, dropout behaviors, zero-division handling in metric calculations, and early stopping logic. Furthermore, AI assistance was leveraged to expand the project’s optimizer support by adding RMSprop, Adagrad and other options, and to integrate PyTorch learning-rate schedulers while keeping the configuration validator aligned with the implementation.

Throughout the development of the data engineering pipeline and the Linear baseline model, Nguyễn Hạo Thiên utilized Gemini 3.8 Flash primarily to guide exploratory analysis design, formalize mathematical leakage controls, and verify data pipeline contracts.

Early in the project, Gemini was consulted to clarify the technical scope of Exploratory Data Analysis on Fashion-MNIST, identifying essential pre-training validation checks—such as class balance, pixel intensity bounds, and representative sample grids—and reinforcing the protocol requirement that normalization statistics must be computed dynamically and exclusively from the training partition to prevent data snooping.

During the pipeline implementation phase, Thiên leveraged AI assistance to structure the class-stratified partitioning algorithm in `split.py` using NumPy’s seeded random generator, implementing mathematical assertion checks for disjointness, complete coverage, and zero duplicate indices, together with atomic JSON manifest serialization.

For DataLoader construction in `loaders.py`, Gemini helped structure `collate_samples` for sample-ID provenance tracking, configure reproducible training shuffling with a dedicated seeded generator and worker seeding function, and enforce deterministic evaluation loaders. In later iterations, AI guidance was used to add targeted regression test coverage (commit `b92fae6`) and refactor configuration schemas to safely enforce the local-only `download=False` ingestion guard without breaking existing interfaces.

All code, assertions, batch tensor dimensions $[B, 1, 28, 28]$ (`float32`), and pipeline configurations were independently verified and tested locally by Thiên, with no model weights or benchmark metrics generated by AI.

References and Evidence

This document uses internal evidence identifiers in addition to the academic references:

- **S-A:** Thiên’s supplied “EDA, Dataset, Loaders, Split, Transforms, and Linear Model” text.
- **S-B:** Khoa’s supplied `progress_report (1).md`.
- **S-C:** Khai_Report_Sections_Draft.md; Khai_Report_Integration_Notes.md.
- **R01–R04:** saved configs, environment, metadata, metrics, histories and selected checkpoints identified in Section 3.2; [R01–R04 run archive](#).
- **A-23:** supplied `dlbench_analysis-20260923T051311Z-1-001/dlbench_analysis` package containing per-run validation predictions, confusion/error records, galleries and forward-only benchmark records; [analysis artifact archive](#).
- **D-23:** three user-supplied dirty-tree screenshots received on 23 September, covering Linear, MLP and protocol configurations; [configuration-diff evidence](#). They support the adjusted-run qualification in Section 3.2.1, not a claim of complete source reconstruction.
- **N-TRAIN:** [training and tuning notebook](#) with EDA/normalization outputs, R03/R04 training and validation, completed Linear Optuna search and incomplete saved MLP search output.
- **N-ANALYSIS:** [benchmark analysis notebook](#) with four-run postprocessing output, benchmarks and gallery-generation code. Source and saved outputs were inspected, not executed during integration.
- **Test evidence:** [dated unit-test evidence archive](#) containing the log, revision metadata and skip explanations.
- **Implementation:** [source tree at revision d9f2295](#), especially contracts, data modules, models, engine/trainer, checkpoint, artifacts and analysis.

Part 1 uses S-A; engine/MLP sections use S-B; configuration, reproducibility and analysis use S-C. Methodology, results and error analysis use the saved run records and A-23. The bibliography lists academic sources; evidence identifiers label project artifacts.

A Companion Confusion Matrices and Error Galleries

The same class ordering and count interpretation as Figure 3.6 apply to Figures A.1–A.3. All error galleries use the same highest-confidence-error selection, not random sampling. Underlying predictions and counts are retained in the corresponding evidence directories.

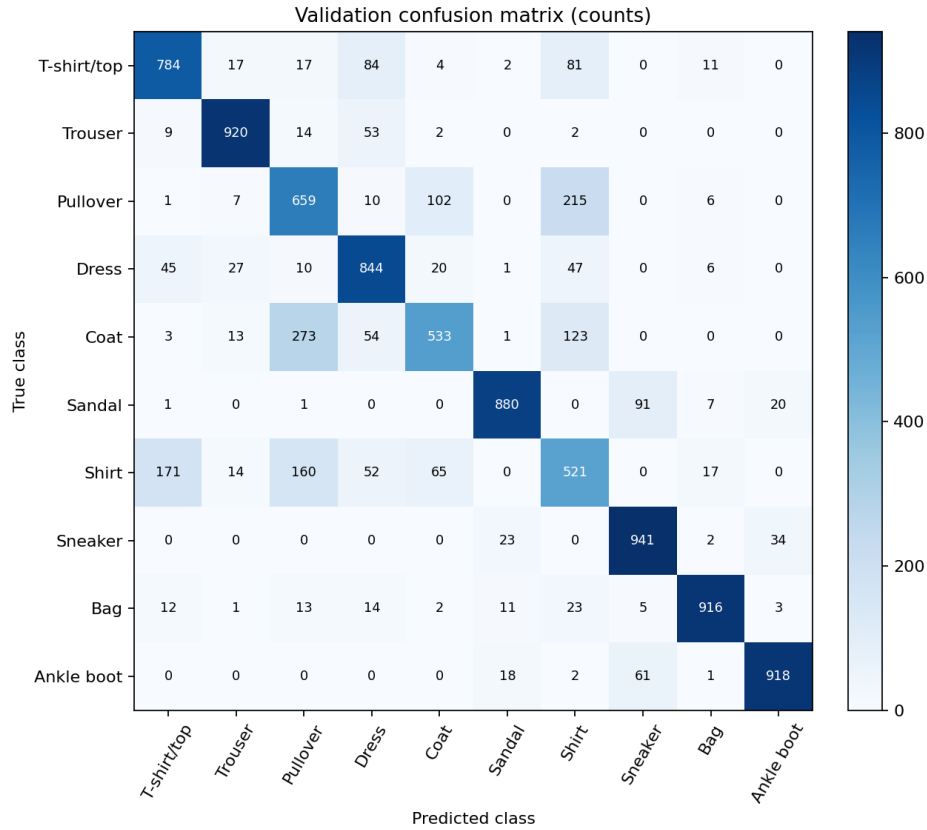


Figure A.1: R01 baseline Linear validation confusion matrix.

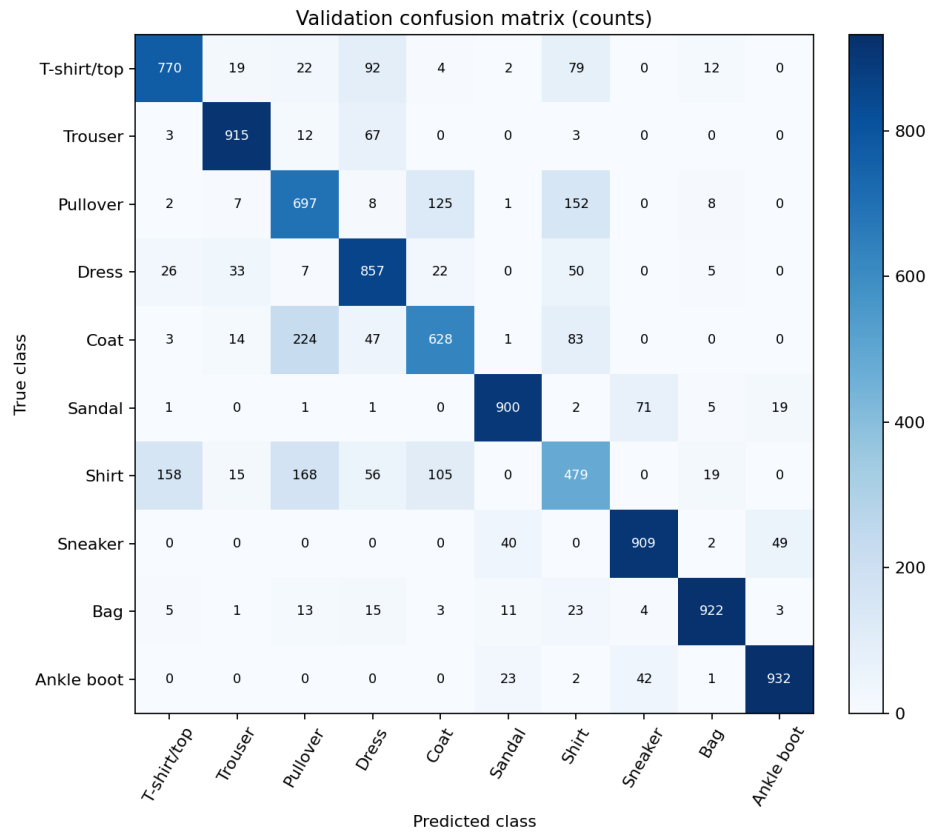


Figure A.2: R03 adjusted Linear validation confusion matrix.

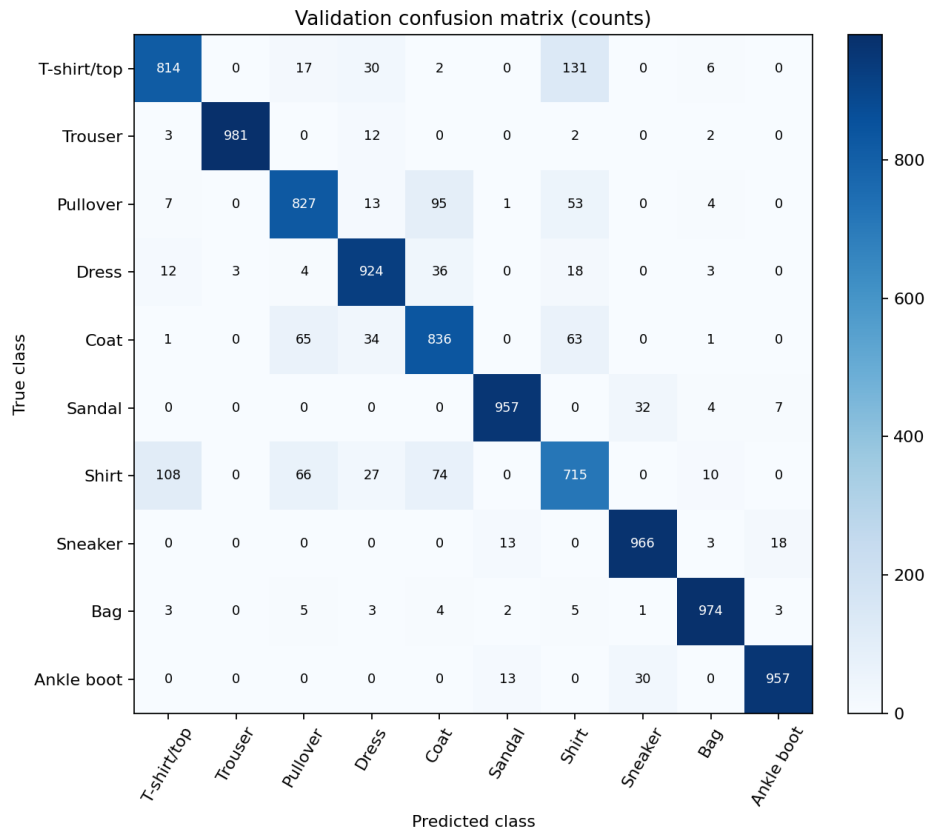


Figure A.3: R04 adjusted MLP validation confusion matrix.

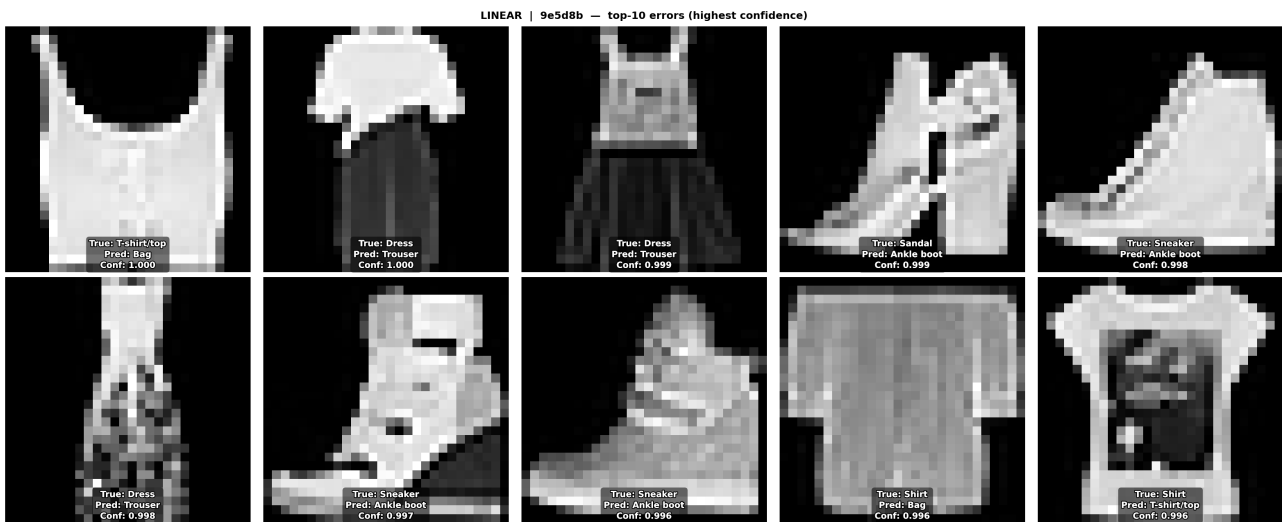


Figure A.4: R01 baseline Linear highest-confidence errors.

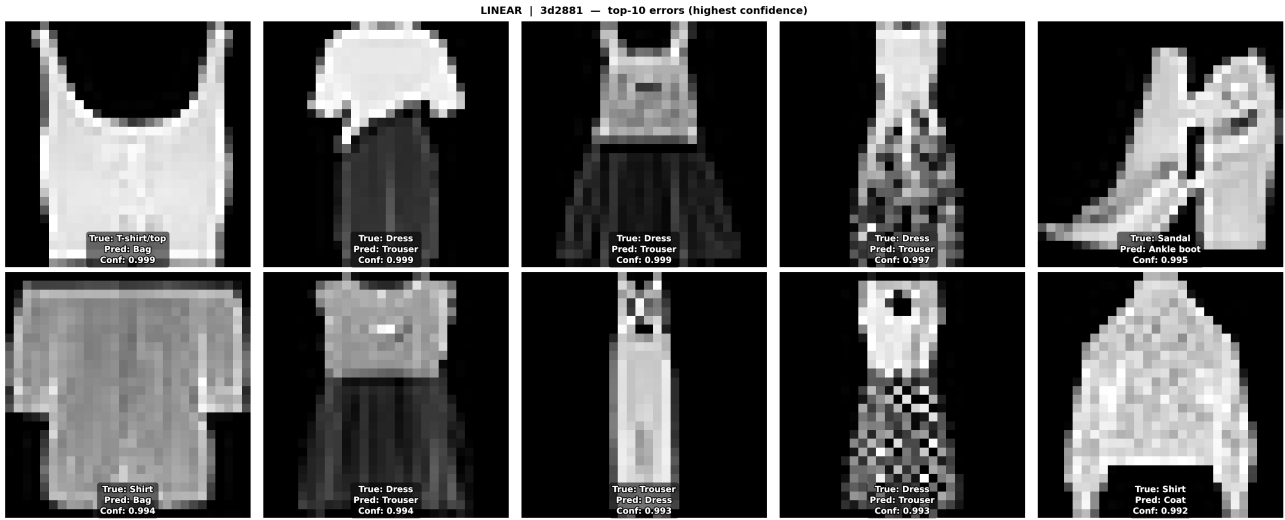


Figure A.5: R03 adjusted Linear highest-confidence errors.

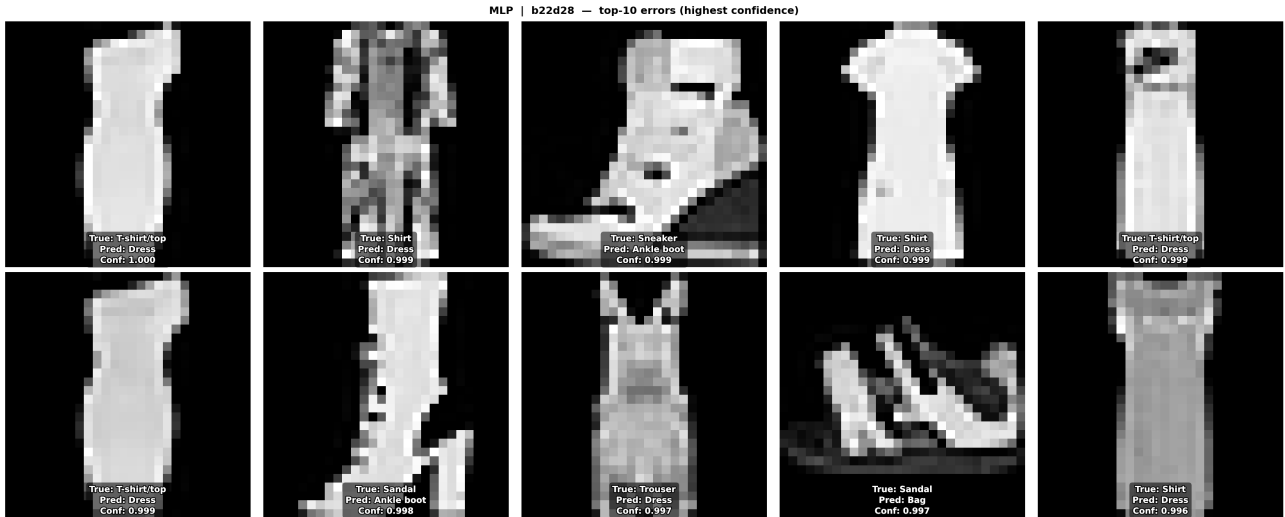


Figure A.6: R04 adjusted MLP highest-confidence errors.

References

- [1] H. Xiao, K. Rasul, and R. Vollgraf. “Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms.” *arXiv preprint arXiv:1708.07747*, 2017.
- [2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. “Gradient-Based Learning Applied to Document Recognition.” *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
- [4] V. Nair and G. E. Hinton. “Rectified Linear Units Improve Restricted Boltzmann Machines.” In *Proceedings of the 27th International Conference on Machine Learning*, pages 807–814, 2010.
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting.” *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [6] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, Cambridge, MA, USA, 2016.
- [7] M. Sokolova and G. Lapalme. “A Systematic Analysis of Performance Measures for Classification Tasks.” *Information Processing & Management*, 45(4):427–437, 2009.
- [8] D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization.” In *3rd International Conference on Learning Representations*, 2015.
- [9] I. Loshchilov and F. Hutter. “Decoupled Weight Decay Regularization.” In *7th International Conference on Learning Representations*, 2019.
- [10] I. Loshchilov and F. Hutter. “SGDR: Stochastic Gradient Descent with Warm Restarts.” In *5th International Conference on Learning Representations*, 2017.
- [11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. “Optuna: A Next-Generation Hyperparameter Optimization Framework.” In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019.